چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۷

کارایی — بخش ۱: رندر، jank، حافظه و پروفایلینگ



اسماعیل مرادی

@esmaeeil-moradi700

چرا کارایی روز هفدهم است؟

کاربر معماری شما را نمی‌بیند — فریم‌های جاافتاده، heap سنگین و گوشی داغ را حس می‌کند. در ۲۰۲۶ سؤال‌های کارایی یک عادت را می‌سنجند: **قبل از حدس زدن، اندازه بگیر**.

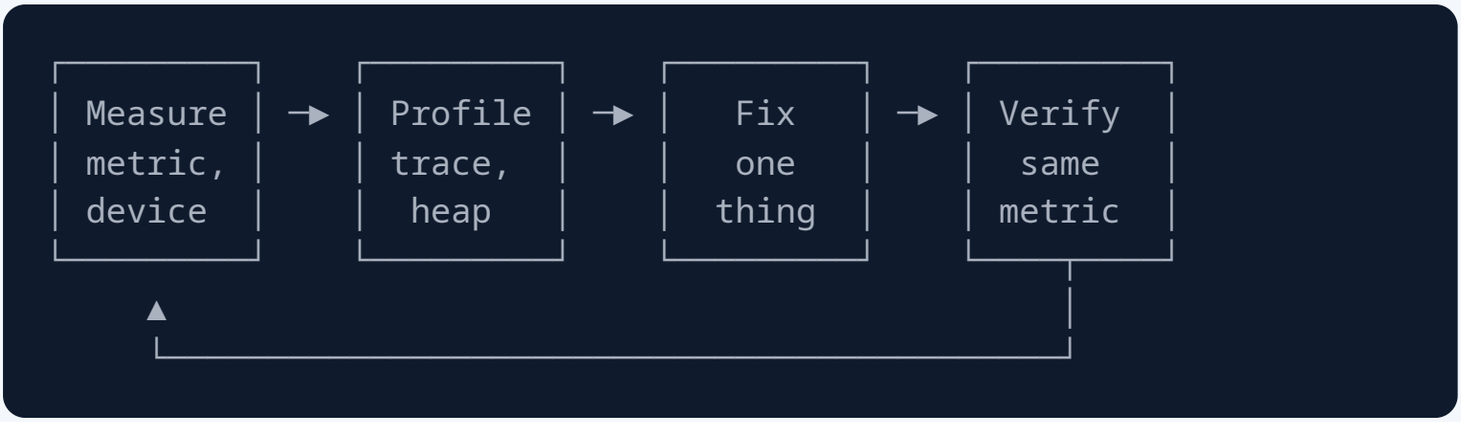
- مهلت هر فریم به نرخ تازه‌سازی صفحه بستگی دارد — ۶۰، ۹۰ یا ۱۲۰ هرتز
- کارایی Compose یعنی **کدام فاز state را می‌خواند**، نه «composable کمتر»
- نشت، churn و bloat سه مشکل متفاوت حافظه‌اند با سه راه‌حل متفاوت
- ابزار trace سیستمی Perfetto است؛ Systrace قدیمی شده
- یک build **شبیه release** را پروفایل کنید و اصلاح را با همان معیار تأیید کنید

سطح	سؤال‌ها
جونیور	jank و خط لولهٔ فریم، مبانی حافظه، overdraw و هزینهٔ layout
مید	فازهای Compose، derived state و لیست‌ها، LeakCanary، پروفایلرهای Studio
سینیور	تحلیل jank با Perfetto، heap و حافظهٔ native، جعبه‌ابزار Compose
آرکیتکت	بودجهٔ ترد اصلی، cache آگاه از حافظه، پروفایلینگ در آزمایشگاه و میدان



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: اندازه‌گیری ← پروفایل ← اصلاح ← تأیید



- **اندازه‌گیری:** اول یک عدد قابل‌حس برای کاربر: فریم‌های janky، زمان فریم در صدک ۹۰، heap بعد از یک مسیر کاربر
- **پروفایل:** برای پیدا کردن علت — معیار می‌گوید *که* کند است، trace می‌گوید *کجا*
- **فقط یک چیز را اصلاح کنید**، بعد روی همان دستگاه و با همان نوع build دوباره اندازه بگیرید

> **قاعدهٔ سرانگشتی**
>
> ادعای کارایی بدون عدد، بدون دستگاه و بدون نوع build فقط یک نظر است. «حس می‌کنم سریع‌تر شده» نتیجه نیست.



اسماعیل مرادی
@esmaeeil-moradi700

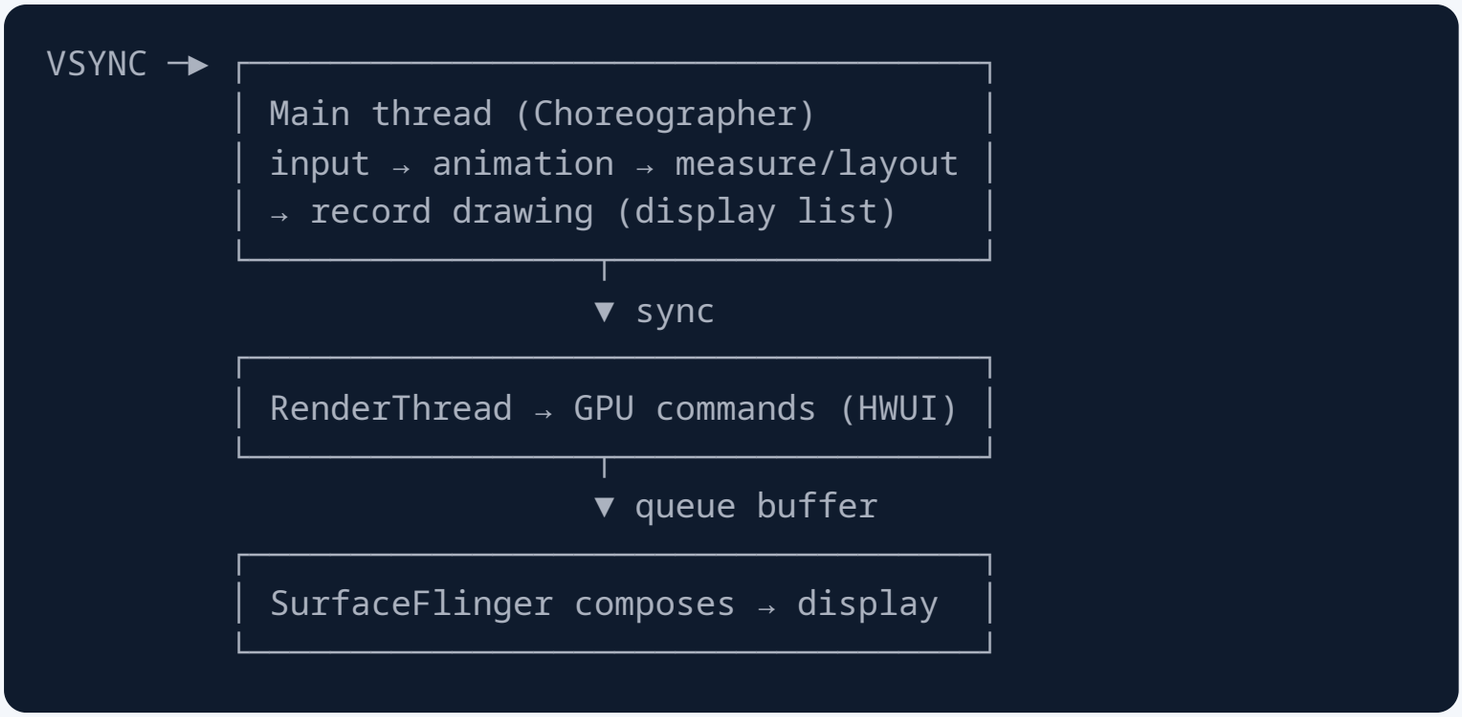
جونیور

jank و خط لولهٔ فریم، مبانی حافظه، overdraw و هزینهٔ layout



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. jank چیست؟ خط لولهٔ فریم



- **ترد اصلی** ورودی، انیمیشن و layout را اجرا می‌کند و بعد دستورهای رسم را فقط *ضبط* می‌کند
- **RenderThread** آن‌ها را به کار GPU تبدیل می‌کند؛ **SurfaceFlinger** همهٔ لایه‌ها را روی صفحه ترکیب می‌کند
- **jank** یعنی فریمی که مهلتش را از دست می‌دهد، پس فریم قبلی روی صفحه می‌ماند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. بودجه به نمایشگر بستگی دارد

نرخ تازه‌سازی	مهلت هر فریم
۶۰ هرتز	حدود ۱۶٫۷ میلی‌ثانیه
۹۰ هرتز	حدود ۱۱٫۱ میلی‌ثانیه
۱۲۰ هرتز	حدود ۸٫۳ میلی‌ثانیه

- هم کار ترد اصلی و هم کار RenderThread برای یک فریم باید در این مهلت جا شوند، وگرنه فریم دیر می‌رسد
- **frozen frame** — بیشتر از ۷۰۰ میلی‌ثانیه در Android vitals — مثل هنگ کردن حس می‌شود، نه یک لرزش کوچک

تلهٔ مصاحبه

«*بودجهٔ هر فریم ۱۶ میلی‌ثانیه است.*» — فقط در ۶۰ هرتز. روی گوشی ۱۲۰ هرتزی همان کد نصف این زمان را دارد. به فریم فکر کنید و با نرخ تازه‌سازی واقعی دستگاه تست کنید.

تلهٔ مصاحبه

«*ترد اصلی پیکسل‌ها را می‌کشد.*» — ترد اصلی فقط دستورهای رسم را ضبط می‌کند؛ RenderThread و GPU آن‌ها را به پیکسل تبدیل می‌کنند. یک `onDraw` یا `drawBehind` سنگین از وقت ترد اصلی می‌خورد؛ لایه‌های بزرگ و افکت‌های سنگین از وقت RenderThread و GPU.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. مبانی حافظه: نشت، churn، bloat

مشکل	چه می‌بینید	راه‌حل رایج
نشت (leak)	با هر تکرار یک مسیر، heap بزرگ‌تر می‌شود؛ صفحه‌های قدیمی زنده می‌مانند	ارجاعی را که از مالکش بیشتر عمر می‌کند اصلاح کنید (روز ۱)
churn	نمودار دندان‌اره‌ای، GCهای پشت‌سرهم هنگام اسکرول یا انیمیشن	در مسیرهای داغ شیء نسازید: `onDraw`، `onBindViewHolder`، composition
bloat	heap از همان اول بالاست	اندازهٔ cacheها را محدود کنید، bitmapها را کوچک decode کنید، دادهٔ نمایش‌داده‌نشده را نگه ندارید

- heap مربوط به Java و Kotlin برای هر اپ سقف دارد (`Runtime.getRuntime().maxMemory()`)؛ عبور از آن یعنی `OutOfMemoryError`
- حافظهٔ native زیر این سقف نیست، اما باز هم بخشی از ردپای حافظهٔ پروسس است که سیستم هنگام کمبود حافظه زیر نظر دارد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. دو افسانه از جواب‌های پارسال

تلهٔ مصاحبه

«*از اندروید ۸ به بعد، bitmapها روی heap جاوا هستند.*» — برعکس است. از Android 3.0 تا 7.1 داده‌های پیکسل روی heap جاوا بودند؛ **از Android 8.0 یعنی API 26 به بعد، در heap مربوط به native هستند**. اپی که پر از bitmap است ممکن است در نمودار heap جاوا خوب به نظر برسد و باز هم حافظهٔ زیادی مصرف کند.

تلهٔ مصاحبه

«*GC هم مثل G1 و CMS در JVM، ترد اصلی را فریز می‌کند.*» — ART همان HotSpot نیست. از اندروید ۸ یک collector از نوع **concurrent copying** دارد با مکث‌های بسیار کوتاه. churn باز هم ضرر دارد: کار GC از CPU و باتری می‌خورد و با فریم‌های شما رقابت می‌کند.

سؤال تکمیلی

«*چرا اپ من بدون هیچ OOM در background کشته می‌شود؟*» — این کار low-memory killer است که پروسس‌های cached را پس می‌گیرد (روز ۱). ردپای کوچک‌تر کمک می‌کند، اما باز هم باید state را درست بازیابی کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. overdraw: هر پیکسل را یک بار رنگ کنید

در Developer options گزینهٔ **Debug GPU overdraw**:

رنگ	معنی
رنگ واقعی	یک بار رسم شده
آبی	یک بار overdraw
سبز	دو بار overdraw
صورتی	سه بار overdraw
قرمز	چهار بار یا بیشتر

- اصلاح: پس‌زمینه‌هایی را که کسی نمی‌بیند حذف کنید (window، root و card همه یک رنگ را می‌کشند)؛ لایه‌های مات تمام‌صفحه را روی هم نچینید
- گزینهٔ **Profile HWUI rendering** برای هر فریم یک ستون در برابر خط مهلت می‌کشد — یک بررسی سریع، نه تحلیل
- هر دو برای Viewها **و** Compose کار می‌کنند، چون هر دو از یک خط لولهٔ رندر HWUI استفاده می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. هزینهٔ layout در Viewها و در Compose

- **در Viewها:** هزینه از measureهای تکراری می‌آید. `RelativeLayout` و `LinearLayout` وزن‌دار ممکن است فرزندها را دو بار measure کنند؛ تودرتو کردنشان این هزینه را چند برابر می‌کند.
- **در Compose:** هر فرزند **در هر pass فقط یک بار** measure می‌شود — measure دوباره exception می‌دهد. intrinsic measurement استثنای صریح است. تودرتو بودن ارزان است.

تلهٔ مصاحبه

«*ConstraintLayout همیشه سریع‌تر است — همه چیز را تخت کن.*» — measure دوباره را حذف می‌کند، اما حل‌کننده‌اش مجانی نیست. یک `LinearLayout` کم‌عمق و بدون وزن ممکن است ارزان‌تر باشد. با trace تصمیم بگیرید، نه از روی عادت.

تلهٔ مصاحبه

«*علت اصلی jank، overdraw است.*» — روی GPUهای امروزی به‌ندرت. اول سراغ کار ترد اصلی بروید: inflate، bind، تخصیص حافظه، I/O دیسک، رقابت روی lock.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

فازهای Compose، derived state و لیست‌ها، LeakCanary، پروفایلرهای Studio



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. در Compose، state را تا جای ممکن دیر بخوانید

فاز	به چه جواب می‌دهد	خواندن state در این فاز دوباره اجرا می‌کند
Composition	چه چیزی نمایش داده شود	composition، layout و drawing
Layout	*کجا*: measure و place	layout و drawing
Drawing	*چه شکلی* باشد	فقط drawing

- نسخه‌های lambdaدار modifierها state را در فاز بعدی می‌خوانند: `offset` و `graphicsLayer` و `drawBehind` و `layout`
- مقدارهایی که در هر فریم عوض می‌شوند (اسکرول، انیمیشن) اگر فقط چیزی را جابه‌جا، کم‌رنگ یا رنگی می‌کنند، هرگز نباید در composition خوانده شوند

تلهٔ مصاحبه

«*composable کمتر یعنی UI سریع‌تر.*» — composableها ارزان‌اند. چیزی که گران است، اجرای دوبارهٔ یک scope بزرگ در هر فریم است. scopeها را کوچک کنید و خواندن را عقب بیندازید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. عقب انداختن خواندن state در کد

```
// BAD: the animated value is read in composition,
// so the Box recomposes on every animation frame
val offsetY by animateDpAsState(target)
Box(Modifier.offset(y = offsetY))

// GOOD: read in the layout phase, no recomposition
val offsetPx by animateFloatAsState(targetPx)
Box(Modifier.offset { IntOffset(0, offsetPx.roundToInt()) })

// GOOD: read in the draw phase, no layout either
val color by animateColorAsState(targetColor)
Box(Modifier.fillMaxSize().drawBehind { drawRect(color) })
```

با `by`، خود `State` همان جایی خوانده می‌شود که متغیر استفاده می‌شود — داخل lambda، یعنی در فاز بعدی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. derived state و لیست‌های lazy

```
// Recomposes only when the Boolean flips
val showToTop by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}

LazyColumn(state = listState) {
    items(
        items = posts,
        key = { it.id },           // stable identity
        contentType = { it.kind }, // reuse per type
    ) { post -> PostRow(post) }
}
```

- `key` : درج و جابه‌جایی، state آیتم‌ها را حفظ می‌کند و همهٔ ردیف‌ها را recompose نمی‌کند
- با `contentType` ، composition آیتم‌ها فقط بین آیتم‌های هم‌نوع دوباره استفاده می‌شود
- مرتب‌سازی و فیلتر کردن جایش در ViewModel است، نه در بدنهٔ composable



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. سه اشتباه کلاسیک در Compose

تلهٔ مصاحبه

`derivedStateOf` همه‌جا. فقط وقتی به‌صرفه است که خروجی **کمتر از** ورودی‌هایش عوض شود. `derivedStateOf { first + last }` فقط سربار اضافه می‌کند.

تلهٔ مصاحبه

نوشتن رو به عقب (backwards write): نوشتن stateای که در همان composition خوانده‌اید (مثلاً `count++` در بدنه) دوباره invalidateش می‌کند — یک حلقهٔ بی‌پایان recomposition. state را در event handlerها و effectها بنویسید.

تلهٔ مصاحبه

قضاوت دربارهٔ کارایی Compose در **build دیباگ**. اپ‌های debuggable بهینه‌سازی‌های ART را کنار می‌گذارند و سربار دیباگ دارند. build ریلیز را با R8 و Baseline Profile اندازه بگیرید (روز ۱۸).

سؤال تکمیلی

«*هنوز همه‌جا به `@Stable` نیاز دارم؟*» — نه. strong skipping از Kotlin 2.0.20 به‌طور پیش‌فرض روشن است (روز ۸). stability را فقط جایی درست کنید که شمارنده‌ها یا traceها کار هدررفته نشان می‌دهند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. LeakCanary چطور نشت را پیدا می‌کند

```
dependencies {
    debugImplementation(libs.leakcanary.android)
    // optional: fail instrumented tests on leaks
    androidTestImplementation(libs.leakcanary.instrumentation)
}
```

- به کد نیازی نیست. اشیائی را زیر نظر می‌گیرد که باید بمیرند: Activityها و Fragmentهای destroyشده، view فرگمنت بعد از `onDestroyView`، ViewModelهای clearشده، Serviceهای destroyشده
- اگر شیئی ۵ ثانیه بعد و پس از یک GC هنوز در دسترس باشد، **retained** حساب می‌شود. وقتی تعدادشان زیاد شد، LeakCanary از heap نسخه برمی‌دارد و **Shark** کوتاه‌ترین مسیر ارجاع قوی از یک GC root را پیدا می‌کند
- اشیای خودتان را هم می‌توانید زیر نظر بگیرید: `AppWatcher.objectWatcher.expectWeaklyReachable(obj, "reason")`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. خواندن leak trace

```
┬───
│ GC Root: System class
│
├─ com.example.SessionManager class
│    Leaking: NO (a class is never leaking)
│    ↓ static SessionManager.listener
│                            ~~~~~~~~
├─ com.example.FeedFragment$onViewCreated$1 instance
│    Leaking: UNKNOWN
│    ↓ FeedFragment$onViewCreated$1.this$0
│                                   ~~~~~~
╰→ com.example.FeedFragment instance
     Leaking: YES (Fragment#onDestroy() was called)
```

- از GC root به پایین بخوانید. ارجاع خراب جایی بین آخرین `Leaking: NO` و اولین `Leaking: YES` است؛ مظنون‌ها با `~~~` زیرخط خورده‌اند
- اینجا یک singleton، listenerای را نگه داشته که Fragment را capture کرده (`this$0`). اصلاح: در `onDestroyView` آن را unregister کنید، یا یک `Flow` منتشر کنید و با `viewLifecycleOwner` جمعش کنید

تلهٔ مصاحبه

«*LeakCanary نشت حافظهٔ native را هم پیدا می‌کند.*» — فقط heap مربوط به Java و Kotlin را تحلیل می‌کند. برای رشد حافظهٔ native، تخصیص‌های native را در Android Studio یا Perfetto ضبط کنید (heapprofd).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. پروفایلرهای Android Studio: ابزار را از روی نشانه انتخاب کنید

نشانه	از اینجا شروع کنید
اسکرول یا انیمیشن janky	trace سیستمی (Perfetto)
CPU بالا، یک متد داغ	نمونه‌برداری callstack از CPU
حافظه مدام بالا می‌رود	LeakCanary، بعد heap dump
GCهای پشت‌سرهم	ردگیری تخصیص‌های Java و Kotlin
رشد حافظهٔ native	ضبط تخصیص‌های native
طوفان recomposition	شمارندهٔ recomposition در Layout Inspector

- برای CPU و زمان‌بندی، یک **build ریلیز profileable** را پروفایل کنید (گزینهٔ «Profile with low overhead»). buildهای debuggable بسیاری از بهینه‌سازی‌های ART را خاموش می‌کنند — زمان‌هایشان دروغ می‌گوید
- شکار نشت در build دیباگ مشکلی ندارد: ارجاع نشت‌کرده به بهینه‌سازی‌های کامپایلر بستگی ندارد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. sampling، tracing و بخش‌های سفارشی

```
<!-- AndroidManifest.xml of the release-like build -->
<application>
    <profileable android:shell="true" />
</application>
```

```
import androidx.tracing.trace

suspend fun loadFeed(): Feed {
    val body = api.fetchFeed()
    return trace("Feed.parse") {
        json.decodeFromString<Feed>(body)
    }
}
```

- بخش‌های شما در traceهای Perfetto و Studio دیده می‌شوند؛ Macrobenchmark هم می‌تواند آن‌ها را اندازه بگیرد (روز ۱۸)
- **method tracing** هر فراخوانی را instrument می‌کند: تعدادها درست‌اند، زمان‌ها متورم. **sampling** زمان‌های واقعی‌تر می‌دهد اما ممکن است فراخوانی‌های کوتاه را نبیند

تلهٔ مصاحبه

«بخش‌های سفارشی trace به اندروید ۱۰ به بالا نیاز دارند.» — `android.os.Trace.beginSection` از API 18 وجود دارد. کتابخانهٔ `androidx.tracing` هم تابع کمکی `trace {}` را برای Kotlin اضافه می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

تحلیل jank با Perfetto، heap و حافظهٔ native، جعبه‌ابزار Compose



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. Perfetto: ضبط یک trace از jank

از سه راه ضبط کنید: گزینهٔ *system trace* در Android Studio، اپ **System Tracing** روی خود دستگاه که از Developer options یا کاشی Quick Settings باز می‌شود، یا `adb`:

```
adb shell perfetto -o /data/misc/perfetto-traces/jank \
  -t 10s sched freq gfx view input am wm dalvik
adb pull /data/misc/perfetto-traces/jank
# open the file in ui.perfetto.dev
```

اندروید ۱۲ به بعد **FrameTimeline** دارد: برای هر اپ یک تایم‌لاین *Expected* و یک تایم‌لاین *Actual*، با نوع jank روی هر فریم دیررسیده.

عادت دوران Systrace	۲۰۲۶
`systrace.py`	Perfetto — از طریق Studio، اپ System Tracing یا `perfetto`
«خط سبز یعنی ۱۶ میلی‌ثانیه»	مهلت از نرخ تازه‌سازی پیروی می‌کند
درصد janky در `dumpsys gfxinfo`	یک بررسی سریع؛ *چرایی* را FrameTimeline می‌گوید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. خواندن trace: از فریم دیررسیده تا علت

- از یک فریم دیررسیده در **Actual Timeline** شروع کنید. اگر نوعش *App Deadline Missed* است، سراغ تردهای خودتان بروید. اگر *SurfaceFlinger*... یا *Buffer Stuffing* است، فقط کد شما مقصر نیست
- بعد **وضعیت** ترد اصلی را در همان فریم ببینید:

وضعیت ترد اصلی	معمولاً یعنی
حالت Running با `doFrame` طولانی	کار زیاد: inflate، bind، recomposition
حالت Runnable	منتظر CPU — رقابت، تردهای پرکار زیاد
خواب روی lock یا binder	منتظر یک ترد یا پروسس دیگر
حالت Uninterruptible sleep	I/O مسدودکننده — دیسک روی ترد اصلی

```sql
SELECT name, dur / 1000000 AS ms
FROM slice
WHERE name LIKE 'Choreographer#doFrame%'
ORDER BY dur DESC
LIMIT 5;
```

تلهٔ مصاحبه

«*علت، همان `doFrame` طولانی است.*» — آن نشانه است. فرزندهایش را باز کنید، مثل `inflate` و `RV OnBindView` و `Recomposer:recompose` و `Lock` … `contention on`، و ببینید ترد منتظر چه بوده.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. heap dump: اندازهٔ shallow در برابر retained

- با تسک heap dump در Studio بگیرید، یا با `adb shell am dumpheap <package> /data/local/tmp/app.hprof`
- **shallow size** یعنی خود شیء. **retained size** یعنی حافظه‌ای که اگر این شیء بمیرد GC آزاد می‌کند. بر اساس retained مرتب کنید.
- نشت‌های Activity و Fragment را فیلتر کنید، بعد **ارجاع‌ها** را تا GC root دنبال کنید
- دستور `adb shell dumpsys meminfo <package>` ردپا را تفکیک می‌کند: heap جاوا، heap مربوط به native، گرافیک، کد — و می‌گوید *کدام* حافظه بزرگ شده

تلهٔ مصاحبه

«*heap جاوا خوب است، پس حافظه خوب است.*» — heap جاوا فقط یک برش است. پیکسل‌های bitmap از API 26 به بعد، کتابخانه‌های native و بافرهای گرافیکی بیرون از آن‌اند. `dumpsys meminfo` و PSS را ببینید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. bitmapها و حافظهٔ native

```
Photo  4000 × 3000 px × 4 B (ARGB_8888)
       = 48,000,000 B  ≈ 46 MiB
Thumb  400 × 300 dp on xxhdpi (×3) = 1200 × 900 px
       = 4,320,000 B   ≈ 4 MiB
```

- در اندازهٔ نمایش decode کنید: Coil و Glide خودشان این کار را می‌کنند؛ با `BitmapFactory` از `inSampleSize` استفاده کنید
- `Bitmap.Config.HARDWARE` پیکسل‌ها را فقط در حافظهٔ گرافیکی نگه می‌دارد — ارزان‌تر، اما تغییرناپذیر و غیرقابل‌خواندن برای CPU
- برای رشد حافظهٔ native — مثل bitmap، JNI و codec — ضبط **تخصیص‌های native** در Studio از heapprofd استفاده می‌کند (اندروید ۱۰ به بعد)

سؤال تکمیلی

«*چطور ثابت می‌کنید نشت برطرف شده؟*» — مسیر را چند بار تکرار کنید، GC را اجبار کنید و heap dump بگیرید: تعداد آن Activity باید به همان تعدادی برگردد که روی صفحه است. بعد یک تست instrumented با LeakCanary نگه دارید تا برطرف‌شده بماند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. جعبه‌ابزار کارایی Compose

ابزار	به چه جواب می‌دهد
شمارندهٔ recomposition در Layout Inspector	کدام composableها و چند بار recompose یا skip می‌شوند
گزارش‌های کامپایلر Compose	کدام پارامترها ناپایدارند، کدام تابع‌ها skippable
composition tracing با `runtime-tracing`	نام composableها داخل traceهای سیستمی
متریک `FrameTimingMetric` در Macrobenchmark	آیا اصلاح، زمان فریم را تغییر داد؟ (روز ۱۸)

```
// build.gradle.kts, Compose Compiler Gradle plugin
composeCompiler {
    reportsDestination = layout.buildDirectory.dir("compose")
    metricsDestination = layout.buildDirectory.dir("compose")
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. استفادهٔ درست از عددها

- تایپ‌هایی که از ماژول‌های بدون کامپایلر Compose می‌آیند ناپایدار حساب می‌شوند؛ یک **فایل پیکربندی stability** می‌تواند آن‌ها را پایدار اعلام کند — یعنی قول می‌دهید تغییر نمی‌کنند
- مقدارهای پرتغییر را به شکل lambda پاس بدهید (`() -> Float`) تا فقط همان جایی که می‌خواند recompose شود

تلهٔ مصاحبه

«*شمارندهٔ recomposition بالا حتماً باید اصلاح شود.*» — فقط اگر فریم‌ها آسیب ببینند. شمارنده‌ها مسیر جست‌وجو را نشان می‌دهند؛ زمان فریم تصمیم می‌گیرد.

تلهٔ مصاحبه

«*برای اطمینان همه چیز را* `@Immutable` *کن.*» — این یعنی دروغ گفتن به کامپایلر. اگر شیء واقعاً تغییر کند، UI به‌روزرسانی را skip می‌کند و دادهٔ کهنه نشان می‌دهد.

سؤال تکمیلی

«*آیا strong skipping موضوع stability را بی‌اهمیت کرد؟*» — نه. پارامترهای ناپایدار با instance مقایسه می‌شوند (`===`)، پس لیستی که در هر emission نو اما برابر است باز هم recompose می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

بودجهٔ ترد اصلی، cache آگاه از حافظه، پروفایلینگ در آزمایشگاه و میدان



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. بودجهٔ ترد اصلی به‌عنوان تصمیم معماری

سهم ترد اصلی: ورودی، تبدیل state به UI، layout و ضبط رسم. بقیهٔ کارها هر کدام جای خودشان را دارند:

کار	جایش
دیسک، دیتابیس، شبکه	APIهای `suspend` در Room و Ktor یا Retrofit — خودشان main-safe هستند
parse، مرتب‌سازی، diff	`Dispatchers.Default`
decode تصویر	تردهای کتابخانهٔ تصویر (Coil، Glide)
راه‌اندازی SDKها	تنبل، در اولین استفاده — نه در `Application.onCreate`
ذخیرهٔ key-value	DataStore به‌جای `SharedPreferences`

- StrictMode در buildهای دیباگ، دیسک و شبکه روی ترد اصلی را می‌گیرد (روز ۱)
- یک قرارداد مشترک برای نام‌گذاری trace، مثل `trace("Feature.step")`، کار همهٔ تیم‌ها را در یک trace از Perfetto قابل‌دیدن می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. «فقط ببرش روی ترد دیگر» کافی نیست

تلهٔ مصاحبه

«*کار را به یک ترد پس‌زمینه ببر، jank تمام می‌شود.*» — کار پس‌زمینه باز هم به فریم‌ها آسیب می‌زند وقتی: برای چند هستهٔ سریع CPU رقابت می‌کند، lockای را نگه می‌دارد که ترد اصلی لازم دارد (که در trace به شکل *Lock contention* دیده می‌شود)، نتیجه‌ها را یک‌جا و انفجاری می‌فرستد و کل صفحه را invalidate می‌کند، یا آن‌قدر شیء می‌سازد که GC را مشغول نگه دارد.

تلهٔ مصاحبه

«*App Startup تابع‌های راه‌اندازی را موازی و روی ترد پس‌زمینه اجرا می‌کند.*» — آن‌ها را **روی ترد اصلی و یکی پس از دیگری** و به ترتیب وابستگی اجرا می‌کند. بردش این است که یک ContentProvider جای چندتا را می‌گیرد.

تلهٔ مصاحبه

«`SharedPreferences.apply()` *ناهمگام است، پس بی‌خطر است.*» — خود مستندات اندروید می‌گوید `apply()` می‌تواند ترد UI را روی `fsync()` مسدود کند. DataStore را ترجیح دهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. cache آگاه از حافظه

```
class ThumbnailCache(maxBytes: Int) :
    LruCache<String, Bitmap>(maxBytes), ComponentCallbacks2 {

    override fun sizeOf(key: String, value: Bitmap): Int =
        value.allocationByteCount

    override fun onTrimMemory(level: Int) {
        when {
            level >= TRIM_MEMORY_BACKGROUND -> evictAll()
            level >= TRIM_MEMORY_UI_HIDDEN ->
                trimToSize(maxSize() / 2)
        }
    }
    // onConfigurationChanged / onLowMemory: omitted
}
// appContext.registerComponentCallbacks(cache)
```

- اندازهٔ cache را بر حسب **بایت** بگیرید، نه تعداد آیتم؛ بودجه را از `ActivityManager.memoryClass` دربیاورید
- اندروید ۱۴ (API 34) سطح‌های `TRIM_MEMORY_RUNNING_*` را deprecated کرد — اپ‌ها دیگر آن‌ها را دریافت نمی‌کنند. به `UI_HIDDEN` و `BACKGROUND` واکنش نشان دهید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. طول عمر، یک تصمیم حافظه‌ای است

- دادهٔ صفحه در ViewModel؛ دادهٔ اپ در repositoryهایی که پشتشان دیسک است — نه در singletonها
- **یک** pipeline تصویر برای کل اپ (یک `ImageLoader` از Coil)، با یک بودجهٔ حافظه
- لیست‌های بزرگ: با Paging 3 فقط یک پنجره نگه دارید (روز ۶)، هرگز کل داده را

تلهٔ مصاحبه

«`android:largeHeap="true"` *مشکل OOM را حل می‌کند.*» — روی بعضی دستگاه‌ها سقف heap را بالا می‌برد، نشت را پنهان می‌کند و GC و کشته شدن در background را پرهزینه‌تر می‌کند. ردپا را اصلاح کنید.

تلهٔ مصاحبه

«`@Singleton` *ش کن که سریع باشد.*» — singleton به اندازهٔ عمر پروسس زنده است: هر cache و listener داخلش عملاً حافظهٔ دائمی می‌شود.

تلهٔ مصاحبه

«*همه‌جا* ART — «*object pool.* اشیای کوتاه‌عمر را ارزان تخصیص می‌دهد. فقط جایی pool کنید که پروفایلر در یک مسیر داغ مثل `onDraw` نشان churn نشان دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. روشی برای پروفایلینگ که در مصاحبه دوام می‌آورد

۱. **بازتولید** روی یک دستگاه واقعی متوسط یا ضعیف، با buildی شبیه release، با R8 و profileable

۲. **اندازه‌گیری** مسیر — با Macrobenchmark یا FrameTimeline — و یادداشت عدد پایه

۳. **trace** برای پیدا کردن علت؛ هر بار فقط یک فرضیه

۴. **اصلاح**، اندازه‌گیری دوباره با همان معیار، و نگه داشتن benchmark به‌عنوان تست regression (روز ۱۸)

۵. **تأیید در میدان:** JankStats، Android vitals، traceهای `ProfilingManager` (اندروید ۱۵ به بعد)

تلهٔ مصاحبه

«عددهای آزمایشگاه همان چیزی است که کاربر حس می‌کند.» — آزمایشگاه تکرارپذیر است؛ میدان حقیقت است. ترکیب دستگاه‌ها، وضعیت حرارتی، بار پس‌زمینه و حجم داده فرق می‌کند. به هر دو نیاز دارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. jank میدانی همراه با زمینه: JankStats

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent { FeedScreen() }

    PerformanceMetricsState
        .getHolderForHierarchy(window.decorView)
        .state?.putState("screen", "feed")

    jankStats = JankStats.createAndTrack(window) { frame ->
        if (frame.isJank) reporter.record(
            durationNs = frame.frameDurationUiNanos,
            states = frame.states.map { it.key to it.value },
        )
    }
}
// onResume/onPause: jankStats.isTrackingEnabled = true/false
```

- listener برای هر فریم اجرا می‌شود و شیء `FrameData` دوباره استفاده می‌شود — آنچه لازم دارید کپی کنید؛ قبل از آپلود نمونه‌برداری و دسته‌بندی کنید
- برچسب‌های UI state هر فریم janky را به یک صفحه نسبت می‌دهند

جواب سینیور در یک جمله

«معیاری قابل‌حس را روی دستگاه واقعی و build ریلیز اندازه بگیر، با trace علت را پیدا کن، یک چیز را اصلاح کن و با همان معیار تأیید کن — در آزمایشگاه و در میدان.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۷ — خلاصهٔ یک‌صفحه‌ای

- مهلت فریم در ۶۰ و ۹۰ و ۱۲۰ هرتز: ۱۶٫۷ و ۱۱٫۱ و ۸٫۳ میلی‌ثانیه.
- ترد اصلی رسم را ضبط می‌کند؛ RenderThread و GPU پیکسل می‌سازند.
- نشت، churn و bloat فرق دارند. پیکسل bitmap از API 26 در heap **native** است.
- GC در ART هم‌روند است — هزینهٔ churn، CPU و باتری است، نه مکث طولانی.
- در Compose، state پرتغییر را در دیرترین فاز بخوانید، مثلاً با `offset {}` یا `drawBehind {}`.
- `derivedStateOf` فقط وقتی خروجی کمتر عوض می‌شود؛ در لیست lazy هم `key` و هم `contentType`.
- LeakCanary برای نشت heap جاوا؛ حافظهٔ native با heapprofd.
- build ریلیز profileable را پروفایل کنید؛ sampling برای زمان، tracing برای تعداد.
- Perfetto جای Systrace را گرفته: از فریم دیررسیده شروع کنید، بعد وضعیت ترد.
- اندازه‌گیری ← پروفایل ← اصلاح یک چیز ← تأیید با همان معیار.

روز ۱۸ در راه است: کارایی، بخش ۲ — حجم اپ، باتری، Baseline Profile و Macrobenchmark. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700